Ampliación editorial basada en fuentes externas — Cómo distinguir Single-File Bundle y ReadyToRun (R2R) frente a Native AOT en .NET 6/8/9+

En .NET moderno, ver un `.exe` sin cabecera CLR en PE-bear no implica necesariamente que sea **Native AOT** (Single-file deployment y ReadyToRun deployment) *(Hecho confirmado)*:

Formato de publicación	Estructura del PE	¿Conserva IL y metadatos?	Cómo analizarlo
Single-File Deployment (`PublishSingleFile`)	`apphost.exe` nativo que lleva concatenado un *Bundle* con las `.dll` administradas (marcador `.net bundle` / `hostfxr`).	**Sí**, completos dentro de las DLLs embebidas.	Abrirlo directamente en **ILSpy 8.x/9.x**, que extrae y decompila las DLLs del bundle.
ReadyToRun (R2R) (`PublishReadyToRun`)	PE con cabecera CLR cuyo `ManagedNativeHeader` apunta a `READYTORUN_HEADER` (firma `0x00525452` = `"RTR"`).	**Sí**, conserva el IL original junto al código nativo precompilado.	Se lee como C#/IL normal en ILSpy o dnSpyEx.
Native AOT (`PublishAot`)	PE 100 % nativo compilado con `ilc` , sin JIT ni bundle de IL.	**No hay IL**.	Se analiza como nativo en IDA o Ghidra.

Leer IL y comparar fuente, IL y decompilación

> **Objetivo de la lección**
>
> Al terminar vas a poder leer el IL de un método sencillo, seguir la pila de evaluación instrucción por instrucción y comparar tres vistas del mismo código: la fuente que escribiste, el IL que generó el compilador y lo que reconstruye el decompilador.

Prerrequisitos: runtime y ensamblados .NET. **Duración sugerida:** 4 horas. Estimación aproximada.

IL es una máquina de pila

A diferencia del assembly x86, que usa registros, el IL es una **máquina de pila**: las instrucciones empujan valores a una pila de evaluación y las operaciones los sacan, calculan y empujan el resultado.

```
ldarg.0        -> push a
ldarg.1        -> push b
add            -> pop b, pop a, push (a+b)
```

Eso hace que el IL sea más fácil de leer que el assembly nativo: no hay que seguir registros, solo la pila. El precio es que hay muchas instrucciones `ld*` / `st*`.

El método de ejemplo

Escribí este método en una clase de consola:

```
static bool Check(int a, int b)
{
    if (a > 50 || b < 50)
        return false;
    return (a + b) * 4 == 480;
}
```

Compilalo en **Debug** (sin optimizaciones) y abrilo en ILSpy. El IL va a ser parecido a este (los offsets exactos cambian según el compilador y la versión):

```
.method private hidebysig static bool Check(int32 a, int32 b) cil managed
{
    .maxstack 2

    ldarg.0
    ldc.i4.s 50
    cgt
    brtrue.s L_false

    ldarg.1
    ldc.i4.s 50
    clt
    brtrue.s L_false

    ldarg.0
    ldarg.1
    add
    ldc.i4.2
    shl
    ldc.i4 480
    ceq
    ret

L_false:
    ldc.i4.0
    ret
}
```

Lectura línea por línea

IL	Qué hace	Pila después
`ldarg.0`	Empuja el primer argumento (`a`)	`[a]`
`ldc.i4.s 50`	Empuja la constante 50	`[a, 50]`
`cgt`	Compara: ¿`a > 50`? Empuja 1 o 0	`[a>50]`
`brtrue.s L_false`	Si el tope es distinto de 0, salta a `L_false`	`[]`
`ldarg.1`	Empuja `b`	`[b]`
`ldc.i4.s 50`	Empuja 50	`[b, 50]`
`clt`	¿`b < 50`? Empuja 1 o 0	`[b<50]`
`brtrue.s L_false`	Si es verdadero, salta a `L_false`	`[]`
`ldarg.0`	Empuja `a`	`[a]`
`ldarg.1`	Empuja `b`	`[a, b]`
`add`	Suma	`[a+b]`
`ldc.i4.2`	Empuja 2	`[a+b, 2]`
`shl`	Desplaza a la izquierda 2 bits (×4)	`[(a+b)*4]`
`ldc.i4 480`	Empuja 480	`[(a+b)*4, 480]`
`ceq`	¿Son iguales? Empuja 1 o 0	`[resultado]`
`ret`	Devuelve el tope	`[]`
`L_false: ldc.i4.0`	Empuja 0 (false)	`[0]`
`ret`	Devuelve 0	`[]`

Comprobación con `a = 40`, `b = 80`:

1. `40 > 50` → 0. No salta.
2. `80 < 50` → 0. No salta.
3. `40 + 80 = 120`.
4. `120 << 2 = 480`.
5. `480 == 480` → 1 (`true`).

El método devuelve `true`. Con `a = 60`, el primer `cgt` da 1 y salta a `L_false`: devuelve `false` sin evaluar el resto.

> `cgt`, `clt`, `ceq`
>
> Son las comparaciones de IL: *compare greater than*, *compare less than*, *compare equal*. Empujan 1 si la condición se cumple y 0 si no. No modifican flags como `CMP`; el resultado es un valor en la pila.

Las tres vistas del mismo método

Fuente (lo que escribiste):

```
if (a > 50 || b < 50)
    return false;
return (a + b) * 4 == 480;
```

IL (lo que generó el compilador): el listado de arriba, con `brtrue` y una etiqueta `L_false`.

Decompilación (lo que reconstruye ILSpy):

```
private static bool Check(int a, int b)
{
    if (a <= 50 && b >= 50)
    {
        return (a + b) * 4 == 480;
    }
    return false;
}
```

Las tres son equivalentes, pero **no son iguales**. El decompilador invirtió la condición (`a > 50 || b < 50` se convirtió en `a <= 50 && b >= 50` por De Morgan), movió el `return false` al final y agregó llaves. Eso es normal: reconstruye una lógica equivalente, no el texto original.

Cuando analices un binario ajeno, no esperes "el código fuente". Esperá **una versión legible y equivalente**, que puede diferir en estilo, nombres y estructura.

Metadatos: dónde están los nombres

En ILSpy, además del código, podés ver:

- **Assembly**: nombre, versión, cultura, referencias (`System.Runtime`, `System.Console`).
- **Namespace** y **Type**: `HolaNet.Program`.
- **Method**: nombre, firma, atributos (`private`, `static`).
- **Tokens**: identificadores numéricos que el runtime usa para referenciar tipos y métodos (por ejemplo, un `MethodDef` con su token). No los necesitás para leer, pero explican por qué el decompilador puede reconstruir las referencias.

Si el binario está ofuscado, los nombres pueden ser `\u0001`, `A`, `B`, `method_0`. La estructura sigue ahí; lo que se pierde es la legibilidad.

Debug vs Release

Compilá el mismo método en Release y compará:

- En Debug, las variables locales y los puntos de secuencia están pensados para depurar; el IL es más "literal".
- En Release, el compilador puede inline-ar métodos chicos, reordenar, eliminar variables y simplificar condiciones.
- Las líneas y nombres de variables locales pueden desaparecer si no hay PDB.

Para aprender, usá Debug. Para analizar un binario real, asumí Release y no te sorprendas si el decompilador muestra un código más raro.

Ofuscación: qué cambia y qué no

Técnica	Qué hace	Cómo se reconoce
Renombrado	Cambia nombres por símbolos ilegibles	`\u0001`, `A`, `method_0`
Cifrado de strings	Guarda las cadenas cifradas y las descifra en runtime	Strings ausentes o ilegibles; llamadas a un método de descifrado
Flujo falso	Agrega saltos y bloques que nunca se ejecutan	Grafos con muchos bloques muertos
State machine	Convierte métodos en una máquina de estados	`switch` sobre un estado, `MoveNext`

El decompilador sigue siendo útil, pero hay que **verificar**. Una condición reconstruida puede ser incorrecta si el flujo está ofuscado. La regla es la misma que en nativo: contrastar estático con dinámico, sobre un programa propio o autorizado.

Práctica

1. Compilá el método `Check` en Debug y en Release. Pegá las dos versiones de IL y marcá tres diferencias.
2. Escribí un método con un `for` que sume 1..10. Encontrá en el IL el salto que vuelve al inicio del bucle.
3. Escribí una propiedad `public int Edad { get; set; }` y buscá el getter y el setter en ILSpy. ¿Qué métodos genera el compilador?
4. Escribí un método `async` que espere un segundo e imprima algo. Buscá la máquina de estados que genera el compilador. No hace falta entenderla entera: identificá `MoveNext`.
5. Compará la decompilación con tu fuente. Marcá al menos una diferencia de estilo y explicá por qué es equivalente.
6. Escribí en tu bitácora: qué método analizaste, qué IL te sorprendió y qué parte no entendés todavía.

Preguntas de repaso

1. ¿IL usa registros? No. Es una máquina de pila: las instrucciones empujan y sacan valores de una pila de evaluación.

2. ¿Qué hace `cgt`? Compara los dos valores del tope: empuja 1 si el primero es mayor que el segundo, 0 si no.

3. ¿Por qué el decompilador muestra una condición distinta de la fuente? Porque reconstruye una lógica equivalente a partir del IL. Puede aplicar De Morgan, reordenar retornos o simplificar expresiones.

4. ¿Qué se pierde en Release? Variables locales, líneas y algunas simplificaciones que el compilador aplica. Los nombres de tipos y métodos se conservan, salvo ofuscación.

5. ¿Qué hace `shl` en el IL? Desplaza a la izquierda la cantidad de bits indicada. `shl 2` equivale a multiplicar por 4.

6. ¿La ofuscación impide analizar un binario .NET? No. Lo hace más difícil. El IL y los metadatos siguen ahí; hay que verificar la reconstrucción con ejecución controlada.

Antes de avanzar

Tenés que poder:

- Leer un método IL de 20 instrucciones y decir qué devuelve.
- Seguir la pila de evaluación con un valor concreto.
- Comparar fuente, IL y decompilación y explicar una diferencia.
- Reconocer señales de ofuscación.

> **Puerta de salida de la Etapa 3**
>
> Si podés compilar tu propio método, leer su IL y explicar qué hace el decompilador, terminaste .NET. La Etapa 4 es el método de análisis completo, con informe.

Fuentes de esta lección

- `Videos y material adicional, Video 38` — MSIL y por qué el código intermedio requiere otras herramientas.
- `Videos y material adicional, Video 40` — .NET Reflector y reconstrucción de un formulario VB.NET.
- `teorias_numeradas_por_etapa.csv` — títulos .NET/dnSpy **pendientes de revisar**.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el video 38 (MSIL) y el video 40 (decompilación VB.NET). **Conocimiento externo:** conjunto de instrucciones IL, pila de evaluación y comportamiento de los decompiladores. **Recomendación editorial:** el método `Check`, la tabla línea por línea y los ejercicios.

Etapa 4 · Práctica guiada

> **Objetivo medible**
>
> Analizar programas propios con método, reconstruir un formato o protocolo desde muestras, contener la ejecución y escribir un informe reproducible.

Prerrequisitos: Etapas 0 a 3. **Duración estimada:** 19 a 25 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
4.1	Método de análisis: de la pregunta al hallazgo	Ciclo de cinco pasos, hipótesis y confirmación.
4.5	Reconstruir formatos y protocolos propios	Deducir campos, longitudes, framing, estados y transformaciones a partir de muestras controladas.
4.2	Reconocer criptografía y encoding en un binario	XOR, base64, constantes de AES/MD5/RC4 y CryptoAPI/CNG.
4.3	El informe reproducible	Estructura, evidencia, límites y divulgación.
4.4	Análisis dinámico seguro: cuándo ejecutar y cuándo no	Matriz de decisión, contención y respuesta.

Recursos de CLS para esta etapa

- Reversing desde cero (ejercicios 1 y 2)
- Inventario y estados de cuarentena

Evaluación de salida

- Analizar un programa propio de punta a punta sin mirar el fuente.
- Reconstruir desde muestras propias un formato binario o un framing simple, justificando campos, longitudes y estados.
- Confirmar una condición con breakpoints y valores concretos.
- Escribir un informe con hash, entorno, evidencia y límites.
- Explicar por qué el caso Shelter no es una práctica de principiante.

Entregable de la etapa

Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Parchear antes de entender.
- Ejecutar una muestra dudosa en el host.
- Escribir conclusiones sin evidencia.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Método de análisis: de la pregunta al hallazgo

> **Objetivo de la lección**
>
> Al terminar vas a poder analizar un programa de práctica con un método explícito: formular una pregunta, inspeccionar el binario, proponer una hipótesis, confirmarla con el debugger y escribir una conclusión que otra persona pueda reproducir.

Prerrequisitos: etapas 0 a 3. **Duración sugerida:** 6 horas (4 para el ciclo con `check.exe` y 2 para el formato propio). Estimación aproximada.

Por qué un método y no "ir probando"

Cuando empezás, el debugger invita a tocar botones: F8, F7, breakpoints, parches. Eso funciona para un programa de 20 líneas y fracasa con uno de 20.000. Un método te evita dos trampas:

1. **Perderte en el camino equivocado.** Sin una pregunta, cualquier instrucción parece interesante.
2. **Confundir correlación con explicación.** Ver que un salto se toma no significa entender por qué.

El capítulo 11 lo dice a su manera: primero buscás lo más fácil (las strings, los mensajes), y si no alcanza, cambiás de estrategia. El análisis visual es rápido, pero hay que saber qué se está buscando (`Reversing desde cero, cap. 11`, 00:00:05–00:02:00).

El ciclo de cinco pasos

```
1. Pregunta     -> ¿qué quiero saber?
2. Inventario   -> ¿qué es este archivo?
3. Estático     -> ¿dónde está la lógica?
4. Hipótesis    -> ¿qué creo que hace y por qué?
5. Confirmación -> ¿cómo lo pruebo?
```

Después se escribe la conclusión y se anotan los límites. Si la confirmación falla, se vuelve al paso 3 con una hipótesis nueva. Ese ida y vuelta es el trabajo real.

Paso 1: la pregunta

Una pregunta útil es específica y verificable. No es "¿qué hace este programa?", sino:

- ¿Qué condición tiene que cumplir la entrada para que muestre "Correcto"?
- ¿Qué compara exactamente la instrucción de la dirección `0x401234`?
- ¿De dónde sale el valor que se usa como índice?

Para el programa de práctica de esta lección, la pregunta es: **¿qué pares de números hacen que `check.exe` imprima "Correcto"?**

Paso 2: inventario

Antes de abrir el debugger, respondé:

Pregunta	Herramienta	Respuesta (ejemplo)
¿Qué tipo de archivo es?	Visor de PE	PE32, ejecutable de consola
¿Es nativo o .NET?	Detect It Easy	Nativo (C)
¿Está empacado?	Secciones/imports	No: `.text` normal, imports de `msvcrt`
¿Qué cadenas muestra?	`strings` o IDA	"Primer valor", "Segundo valor", "Correcto", "Incorrecto"
¿Qué imports usa?	Visor de PE	`printf`, `scanf`, `puts`
¿Tiene símbolos?	PDB	Sí, si lo compilaste en Debug

Ese inventario ya te dice mucho: un programa de consola que usa `scanf` y `puts`, sin empaquetar, con cadenas claras. No hace falta empezar por el entry point.

Árbol de decisión de triage inicial («¿Por dónde empiezo?»)

Cuando te enfrentás a un binario desconocido (un crackme, un programa propio compilado meses atrás o un componente de laboratorio), los primeros diez minutos del **Paso 2 (Inventario)** definen qué herramientas abrir y a qué lección del curso acudir. Usar IDA sobre un ensamblado .NET puro o buscar strings en un binario empaquetado con UPX hace perder horas. Este árbol de decisión conecta los tres niveles del curso (Principiante, Intermedio y Avanzado) en un flujo único (y se complementa con el Hilo conductor multi-nivel y kit de plantillas C/C++):


FASE 1 · CONTENEDOR, ARQUITECTURA Y RUNTIME (PE-bear / DIE · 2 minutos)
¿NO empieza con "MZ" / "PE\0\0"?
¿Es archivo de datos / captura de mensajes? ⟶ Lección 4.5 (Formatos y protocolos)
¿Es código máquina sin cabecera (posición independiente)? ⟶ Lección 2.23 (Shellcode)
¿Es PE32 (0x10B, 32 bits) o PE32+ (0x20B, 64 bits)? (Lecciones 1.4 y 2.5)
32 bits (x86): argumentos en stack (cdecl/stdcall) o ECX/EDX (fastcall/thiscall);
en Windows x64 corre bajo WOW64 (ver Lección 2D.2).
64 bits (x64): argumentos en RCX, RDX, R8, R9 + shadow space de 32 bytes.
¿Tiene cabecera CLR (.NET) o marcador ".net bundle" (hostfxr)? (Lección 3.1)
SÍ (IL puro, ReadyToRun "RTR\0" o Single-File Bundle):
Abrirlo en ILSpy / dnSpyEx (Etapa 3: Lecciones 3.1 y 3.2).
NO (Nativo puro o .NET Native AOT):
Avanzar a la FASE 2.
FASE 2 · ENTROPÍA, SECCIONES, IMPORTS Y HUELLA DE COMPILADOR (3 minutos)
¿Entropía alta (> 6,8), secciones renombradas (UPX0/UPX1), permisos W+X
o primera sección con RawSize = 0 e IAT mínima (LoadLibraryA + GetProcAddress)?
SÍ: Posible PACKER (Lección 2.17).
En UPX x86: "ESP trick" (F8 sobre pushad + HW breakpoint en [ESP]).
En UPX x64: breakpoint de memoria en ejecución sobre la sección vacía.
En el OEP: dumpear y reconstruir IAT con Scylla (si es VM comercial: ver 2.10 / 5.4).
IAT casi vacía pero entropía normal:
Resolución dinámica por PEB / API Hashing (Lección 2.14):
buscar lecturas de fs:[30h] / gs:[60h], recorrido de InMemoryOrderModuleList
([rsi+20h] = DllBase en x64) y constantes de hash de 32 bits.
¿Qué compilador o runtime delata la huella? (Lección 2.13)
MSVC C/C++ (Rich header + vcruntime140.dll / ucrtbase.dll + __security_cookie):
¿Hay vtables con RTTI en vtable[-1] o __CxxFrameHandler3/4 en .pdata/.xdata?
⟶ Lección 2.18 (C++ profundo: RTTI, std::string SSO, STL y excepciones).
MFC (mfc140u.dll o estático con CWnd/CDialog):
Lección 2.12: localizar GetMessageMap() ⟶ AFX_MSGMAP y AFX_MSGMAP_ENTRY.
MinGW / GCC (msvcrt.dll + __mingw_* + sección .eh_frame, sin Rich header):
Lección 2.13: ubicar main pasado a __getmainargs.
Delphi (recursos RCDATA con firma "TPF0", TForm/TObject):
Lección 2.13: ABI register en x86 (EAX=Self, EDX, ECX) y longitud en [ptr-4].
Go (binario grande estático, "Go build ID", tabla pclntab):
Lección 2.13: ir a main.main; recordar que string = {ptr, len} sin byte 0.
Rust (rutas "src/main.rs", mensajes de pánico Option::unwrap, símbolos _ZN / _R):
Lección 2.13: separar ramas de pánico del flujo principal; string = {ptr, len}.
FASE 3 · PRE-MAIN, OFUSCACIÓN Y ROL DEL COMPONENTE (5 minutos)
¿El directorio TLS (AddressOfCallBacks) tiene punteros?
SÍ: Inspeccionar TLS Callbacks antes de main (Lección 2.11); suelen ejecutar
chequeos anti-debug (IsDebuggerPresent, PEB.NtGlobalFlag, NtQueryInformationProcess, 2.15).
¿La ventana de Strings (Shift+F12) no muestra los mensajes esperados?
Buscar cadenas UTF-16 (funciones W, Lección 2.7), cadenas armadas en el stack (mov byte)
o bucles de descifrado XOR / RC4 / ChaCha20 / AES previos al uso (Lecciones 2.10 y 4.2).
¿El grafo de la función en IDA/Ghidra es un único bucle gigante con switch?
Distinguir Aplanamiento de Flujo (CFF: bloques con lógica real y variable de estado)
frente a Máquina Virtual (VM: handlers genéricos leídos por VIP) (Lección 2.10).
¿Cuál es el rol del binario y tu objetivo de análisis?
Programa de consola/GUI estándar ⟶ Ciclo de 5 pasos de esta lección (4.1) + 2.6 / 2.7 / 2.16.
Servicio de Windows (StartServiceCtrlDispatcher) ⟶ Lección 2D.4 (Servicios).
Persistencia en Registro / Tareas / WMI ⟶ Lecciones 2D.3, 2D.5 y 2D.9.
Servidor o cliente COM / DCOM / RPC / Named Pipe ⟶ Lecciones 2D.6 a 2D.10 y 4.5.
Diagnóstico de crash, corrupción de heap o comparación con parche ⟶ Lecciones 2.19, 2.20, 2D.11 y 5.2.
Driver de kernel (.sys, importa ntoskrnl.exe / WdfVersionBind) ⟶ Etapa 6 (6.1 a 6.6).

Nivel	Señal observada en los primeros 5 minutos	Qué significa (hipótesis inicial)	Acción inmediata y lección de referencia
Principiante	Imports claros (`scanf`, `printf`, `MessageBoxW`) y strings visibles en `.rdata`	Binario C/C++ sin empaquetar ni ofuscar	Buscar la string clave con `Shift+F12`, saltar por xref (`X`) y aplicar el ciclo de 5 pasos (2.7, 4.1).
Principiante	Las strings de éxito y error están en el **mismo bloque básico** sin salto condicional entre ellas	Selección sin salto (*branchless*) producida por el compilador (`-O2`)	Buscar una instrucción `CMOVcc` (`cmovne` / `cmove`) que elige qué puntero de cadena pasar a `puts` / `printf` (2.4, 2.8, 4.1).
Principiante	Importa `mscoree.dll` (`_CorExeMain`) o contiene metadatos `.NET` / marcador `.net bundle`	Aplicación administrada .NET (IL, ReadyToRun o Single-File)	Abrir en **ILSpy** o **dnSpyEx** para leer C# e IL antes de tocar un desensamblador nativo (3.1, 3.2).
Intermedio	Secciones `UPX0` (`RawSize = 0`) y `UPX1` (entropía > 7,5, permisos `W+X`), pocos imports	Ejecutable empaquetado con UPX	Llegar al OEP (*ESP trick* en x86 o breakpoint de memoria en ejecución sobre `UPX0` en x64), dumpear y reconstruir IAT con Scylla (2.17).
Intermedio	Mi breakpoint en `main` nunca se alcanza, o una variable global ya llega alterada al entrar a `main`	Código ejecutado antes del entry point principal	Revisar el directorio **TLS (`AddressOfCallBacks`)** en PE-bear/IDA y poner breakpoint en cada callback (2.11, 2.15).
Intermedio	Casi sin imports ni strings, pero lee `gs:[60h]` → `[rax+18h]` → `[rax+20h]` y rota enteros (`ror edx, 0Dh`)	Resolución dinámica de APIs por recorrido del PEB y *API hashing*	Poner breakpoint en el `call reg` del resolvedor o cruzar constantes con listas de hashes de exportaciones (2.14).
Intermedio	Constantes `0x637c777b` (AES), `"expand 32-byte k"` (ChaCha20) o `0x9E3779B9` (TEA) en `.rdata`/código	Transformación criptográfica estándar o cifrado de configuración/mensajes	Etiquetar la función criptográfica por su constante mágica e interceptar su entrada/salida en vez de analizar las rondas internas (4.2, 2.21).
Avanzado	Llamadas indirectas `call qword ptr [rax+18h]` donde `rax` proviene de `[rcx]`	Despacho virtual C++ (`vtable`) o interfaz COM (`IUnknown`)	Ubicar la `vtable` en `.rdata`, leer `vtable[-1]` (`Complete Object Locator`, RVAs en x64) para recuperar el nombre de la clase (2.18, 2D.6).
Avanzado	El proceso muere dentro de `ntdll!RtlFreeHeap` lejos de donde ocurrió el error real	Corrupción de heap previa (desborde, *use-after-free* o *double free*)	Activar **Full Page Heap** (`gflags /p /enable app.exe /full`) o grabar traza **TTD** en WinDbg y retroceder con `ba w4` + `g-` (2.19).
Avanzado	Archivo `.sys` que importa `WdfVersionBind` y registra `EvtIoDeviceControl`	Driver de modo kernel basado en KMDF	Desglosar cada constante `CTL_CODE` del `switch` (DeviceType, Access, Function, Method), revisar `SDDL` y auditar tamaños mínimos y usos de `METHOD_NEITHER` (6.2, 6.4).

Paso 3: análisis estático

Abrí el binario en IDA Free o Ghidra y seguí las cadenas:

1. `Shift+F12`, buscá "Correcto".
2. `X` sobre la cadena para ver quién la usa.
3. Leé el bloque: vas a encontrar una comparación y dos caminos.
4. Renombrá: `check`, `primer_valor`, `segundo_valor`, `camino_ok`, `camino_error`.
5. Anotá la condición que deduce el grafo.

En el programa del capítulo 15, el análisis estático muestra:

- `scanf("%d", &a)` y `scanf("%d", &b)`;
- `if (a > 50 || b < 50) error;`
- `if ((a + b) * 4 == 480) correcto; else incorrecto;`

El capítulo 15 hace exactamente este recorrido, con `0x32` (50) y `0x1E0` (480), y lo confirma después en el debugger (`Reversing desde cero, cap. 15`, 00:06:00–00:16:00).

Paso 4: la hipótesis

Una hipótesis se escribe como una afirmación falsable:

> "El programa acepta cualquier par `(a, b)` con `a <= 50`, `b >= 50` y `(a + b) * 4 == 480`. Con `a = 40` y `b = 80` debería imprimir 'Correcto'; con `a = 40` y `b = 70` debería imprimir 'Incorrecto'."

Falsable significa que podés imaginar un resultado que la contradiga. "El programa compara números" no es falsable; "compara contra 50 y contra 480" sí.

Paso 5: confirmación dinámica

Ahora sí, el debugger. El objetivo no es "ver qué pasa", sino **comprobar la hipótesis**.

Preparación

1. Copiá el programa a la VM.
2. Abrilo en x64dbg.
3. Buscá la cadena "Correcto" y seguí la referencia hasta el bloque.
4. Etiquetá `camino_ok` y `camino_error`.
5. Poné un breakpoint en la comparación final (`cmp eax, 0x1E0`).

Ejecución con `40` y `80`

1. `F9` para correr.
2. Ingresá `40` y `80`.

3. El programa se detiene en el breakpoint.
4. Mirá `EAX`: debería ser `0x1E0` (480).
5. Mirá `ZF`: debería ser 1.
6. `F8` para ejecutar el salto: no se toma el salto a `camino_error`.
7. `F9`: imprime "Correcto".

Ejecución con `40` y `70`

1. Reiniciá (`Ctrl+F2`).
2. Ingresá `40` y `70`.
3. En el breakpoint, `EAX` debería ser `0x1B8` (440).
4. `ZF` = 0; el salto se toma; imprime "Incorrecto".

Ejecución con `60` y `80`

1. Reiniciá.
2. Ingresá `60` y `80`.
3. El programa ni siquiera llega a la comparación final: falla la primera condición (`a > 50`).
4. Confirmá dónde se detiene y qué salto se toma.

> **Tu compilador puede generar otro código**
>
> Las direcciones y la forma exacta dependen del compilador, las optimizaciones y la arquitectura. Lo que no cambia es la **lógica**: dos comparaciones de rango, una suma, un desplazamiento y una comparación final. Si tu listado no coincide, buscá esas operaciones, no una dirección mágica.

Dos casos que afinan el método

El ciclo de arriba sirve para la mayoría de los programas, pero conviene conocer dos situaciones que lo ponen a prueba.

La decisión no está donde la buscás (cap. 16). Lo habitual es un bloque "bueno" y un bloque "malo" separados por un salto. Pero un programa puede imprimir el mensaje correcto o el incorrecto **desde el mismo bloque**, eligiendo con un `CMOVcc` (ver la lección de flujo) qué puntero de cadena usar. Entonces no hay un salto evidente para entender —ni para parchear—. La lección de método: si las cadenas de éxito y error están en el mismo bloque, buscá un movimiento condicional, no un `Jcc`. En ese mismo ejercicio aparece un `strlen` *inline* (un bucle que incrementa un puntero hasta el byte `0` y resta inicio de final) en vez de una llamada a la API: reconocer funciones estándar escritas a mano es parte del método (`Reversing desde cero, cap. 16`; transcripción .txt, sin marcas).

Sin símbolos, igual se puede (cap. 20). El capítulo 20 resuelve un crackme **sin PDB**, y es un buen modelo del ciclo: ejecutar y anotar las cadenas; abrir en IDA; buscar la cadena de error con `Shift+F12` y llegar al código con `X`; **pintar** el bloque malo (salmón), el bueno (verde) y el camino (celeste); como la función es chica, analizarla **desde el inicio** hasta la comparación; renombrar funciones y variables mientras se lee,

poniendo `printf?` con signo de pregunta cuando no hay certeza; deducir el algoritmo (ahí, el serial es la suma de los caracteres del nombre) y confirmarlo en el debugger con *popups* de valores (`Reversing desde cero, cap. 20`). Un detalle útil de ese capítulo: en 32 bits, **el primer argumento es el que está más cerca del `call`**; y sobre una variable del stack, `A` la convierte en array/define su tamaño y `H` muestra el valor en decimal.

> **Cuando el resultado depende del tiempo o del entorno**
>
> A veces el valor correcto no es fijo: el capítulo 19 analiza un programa de 64 bits cuya clave se calcula a partir de `GetSystemTime` (depende del minuto actual). Entonces no se puede "ver el valor y volver a ingresarlo", porque para cuando lo tipeás ya cambió. Ahí la confirmación dinámica no alcanza por sí sola: hay que **entender el algoritmo** (o automatizarlo). Es un recordatorio de que estático y dinámico se complementan, y de que una sola corrida no demuestra el caso general (`Reversing desde cero, cap. 19`).

Cuando la pregunta es un formato o un protocolo propio

Este apartado muestra la reconstrucción de **un archivo propio** dentro del ciclo general. La lección 4.5 · Reconstruir formatos y protocolos propios sistematiza el trabajo con muestras, framing, protocolos con estado y validación independiente.

Muchas preguntas reales no son "¿qué condición acepta?" sino "¿cómo es el archivo que guarda este programa?" o "¿qué significan los mensajes que manda?". El ciclo de cinco pasos sirve igual; cambian las herramientas de cada paso. Este apartado es **recomendación editorial** apoyada en **conocimiento externo**; el ejemplo es **ilustrativo** y supone un programa **propio** que guarda su configuración en un archivo binario.

1. Pregunta. Concreta: "¿qué campos tiene el archivo `.blc` y qué valida el programa al abrirlo?".

2. Inventario. Dos o tres archivos generados por el propio programa con datos distintos, y un volcado hexadecimal de cada uno. Las primeras cosas a buscar: bytes fijos al inicio (**número mágico**), números que coinciden con el tamaño del archivo o de un bloque (**campos de longitud**), texto legible (UTF-16 incluido, ver IDA y el análisis estático) y zonas de alta entropía (posible compresión o cifrado, ver criptografía y encoding).

3. Estático: encontrar el parser. El código que lee el formato está detrás de las llamadas que traen los bytes: `ReadFile`, `fread`, `MapViewOfFile` para archivos; `recv` / `WSARecv` para red. Desde la xref, seguí el buffer: cada comparación contra una constante es una validación (el mágico, una versión) y cada lectura `[buffer+N]` con un tamaño fijo es un campo. Una lectura seguida de un salto que usa ese valor como cantidad o desplazamiento es un campo de longitud u offset.

4. Hipótesis: la tabla de campos. Escribí la estructura como tabla, con la evidencia de cada fila:

Offset	Tamaño	Tipo	Significado (hipótesis)	Evidencia
`0x00`	4	bytes	Mágico `BLOC`	Comparado con `0x434F4C42` antes de seguir
`0x04`	2	uint16 LE	Versión	`cmp ax, 2` y rama de error "versión no soportada"
`0x06`	2	uint16 LE	Cantidad de registros	Controla el contador del bucle
`0x08`	4	uint32 LE	Checksum del resto	Comparado con el resultado de una función con tabla de 256 entradas (CRC32)
`0x0C`	variable	registros	Datos	Leídos de a `0x20` bytes dentro del bucle

5. Confirmación diferencial. Con una **copia** del archivo, cambiá **un solo byte** por vez y observá: si cambiar la versión dispara el error de versión, la fila está confirmada; si cambiar un dato hace fallar el checksum, ya sabés que lo cubre. La prueba final es escribir un **lector independiente** (un script de Python con el módulo `struct`, o una descripción declarativa en un lenguaje como Kaitai Struct) que lea los tres archivos y dé los mismos valores que muestra el programa.

Cuidados: trabajá sobre copias; el programa puede reescribir o "reparar" el archivo al abrirlo. Para protocolos de red, el mismo método se aplica a capturas de tu propio cliente y servidor en la VM aislada, sin apuntar a servicios de terceros. Si un campo está cifrado o comprimido, la tabla lo dice así ("bloque cifrado, clave no recuperada") en lugar de inventarle estructura.

Cómo se escribe la conclusión

Una conclusión útil tiene cuatro partes:

1. **Respuesta a la pregunta.** "El programa acepta pares con `a <= 50`, `b >= 50` y `(a+b)*4 == 480`."
2. **Evidencia.** "Breakpoint en `0x...`: con (40,80), `EAX=0x1E0`, `ZF=1`, imprime 'Correcto'; con (40,70), `EAX=0x1B8`, `ZF=0`, imprime 'Incorrecto'."
3. **Método.** "Análisis estático en IDA Free + confirmación en x64dbg; binario compilado con gcc -O0."
4. **Límites.** "No se probaron todos los pares; la conclusión vale para esta compilación."

Eso es un hallazgo reproducible. No es "lo crackeé": es "entendí la condición y la verifiqué".

Errores frecuentes

Error	Por qué pasa	Cómo evitarlo
Buscar solo strings	El programa puede no tener mensajes claros	Probá imports, xrefs y el entry point
Confundir dirección virtual con offset	ASLR y secciones	Usá RVA u offset, o rebaseá
Leer un salto al revés	`jne` salta cuando **no** son iguales	Preguntá qué camino toma el salto
Confundir signo	`JA` vs `JG`	Mirá el tipo del dato y la instrucción
Parchear como primera opción	Es más rápido que entender	Primero explicá; después decidí si hace falta
No anotar	Te olvidás de lo que probaste	Bitácora en el momento

Práctica

1. Analizá tu `check.exe` con el ciclo de cinco pasos. Escribí la pregunta, el inventario, la hipótesis y la confirmación.
2. Probá tres pares válidos y tres inválidos. Anotá los valores de `EAX` y `ZF` en cada caso.
3. Cambiá el programa: hacé que la condición sea `(a + b) * 2 == 240`. Recompilá y repetí el análisis sin mirar el código fuente. ¿Pudiste deducir la nueva constante?
4. Agregá una tercera condición (por ejemplo `a != b`) y analizala.
5. Escribí un párrafo de conclusión con las cuatro partes.
6. Escribí un programa propio que guarde tres o cuatro valores en un archivo binario con un mágico, una versión y un campo de longitud. Sin mirar el fuente, armá la tabla de campos con evidencia, confirmala cambiando un byte por vez sobre copias y escribí un lector en Python con `struct` que dé los mismos valores.

Preguntas de repaso

1. ¿Cuál es la diferencia entre una hipótesis y una observación? La hipótesis es una afirmación que hacés antes de confirmar; la observación es lo que el debugger te muestra. La hipótesis se contrasta con la observación.

2. ¿Por qué conviene empezar por las strings? Porque en programas con interfaz o mensajes, las cadenas te llevan rápido a la lógica y a los bloques de éxito/error.

3. ¿Qué hacés si la confirmación falla? Volvés al análisis estático con una hipótesis nueva. No parcheás para que "funcione": entendés por qué falló.

4. ¿Por qué hay que anotar la compilación y las direcciones? Porque el mismo código compilado con otra versión u optimización puede generar otro listado. Sin esos datos, el informe no es reproducible.

5. ¿Qué significa que un hallazgo sea reproducible? Que otra persona, con el mismo binario y el mismo método, puede obtener el mismo resultado.

6. Querés entender el formato de archivo de un programa. ¿Dónde empezás a buscar el parser? En las xrefs a las APIs que traen los bytes (`ReadFile`, `fread`, `MapViewOfFile`; `recv` para red) y siguiendo el buffer: las comparaciones con constantes son validaciones y las lecturas a offsets fijos, campos.

7. ¿Qué es una confirmación diferencial y por qué se hace sobre copias? Cambiar un solo byte por vez y observar qué validación falla o qué valor cambia. Se hace sobre copias porque el programa puede reescribir el archivo y porque el original es la referencia.

8. ¿Cuándo das por confirmada la tabla de campos? Cuando un lector independiente (un script propio) lee varios archivos y obtiene los mismos valores que muestra el programa; lo que no se pudo explicar queda anotado como tal.

Antes de avanzar

Tenés que poder:

- Formular una pregunta específica y una hipótesis falsable.
- Hacer el inventario de un binario antes de ejecutarlo.
- Confirmar una condición con un breakpoint y valores concretos.
- Escribir una conclusión con evidencia y límites.
- Reconstruir el formato de un archivo propio como tabla de campos con evidencia y confirmarlo con un lector independiente.

> **Puerta de salida**
>
> Si podés analizar un programa propio y explicar la condición sin mirar el fuente, aprobaste el método. La lección siguiente convierte ese trabajo en un informe.

> **Lo que queda fuera**
>
> El objetivo es reconocer y documentar compresión, checksums, hashes, MAC y cifrado, no romper cifrado ni recuperar claves. Tampoco se reconstruyen protocolos de servicios de terceros. La lección 4.5 enseña el procedimiento con formatos y comunicaciones de programas propios; el *fuzzing* guiado por formato se presenta en la optativa 2.24.

Fuentes de esta lección

- `Reversing desde cero, cap. 11` — análisis dinámico, strings, etiquetas y hipótesis.
- `Reversing desde cero, cap. 15` — análisis estático, condiciones `0x32` / `0x1E0` y confirmación.
- `Reversing desde cero, cap. 16` — decisión con `CMOVcc` en el mismo bloque y `strlen` inline.

- `Reversing desde cero, cap. 19` — clave dependiente del tiempo (`GetSystemTime`): por qué no se reproduce ingresando el mismo valor.
- `Reversing desde cero, cap. 20` — modelo del ciclo sin símbolos: strings, xref, pintar, analizar desde el inicio, renombrar con `?`.
- `plantilla_informe.md` — plantilla base para el informe.
- Conocimiento externo: struct — Interpret bytes as packed binary data (documentación de Python) y Kaitai Struct (lenguaje declarativo para describir formatos binarios).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el método y los valores de los capítulos 11 y 15 (`0x32` , `0x1E0` , `40` / `80`), el caso `CMOVcc` / `strlen` inline del 16, el modelo sin símbolos del 20 y la clave por tiempo del 19. **Conocimiento externo:** ciclo hipótesis-confirmación, criterios de reproducibilidad y la reconstrucción de formatos (mágicos, campos de longitud, checksums). **Ejemplo ilustrativo:** el formato `.blc` y su tabla de campos. **Recomendación editorial:** el programa `check.exe` , la tabla de errores y la confirmación diferencial con lector independiente.

Reconstruir formatos y protocolos propios

Objetivo de la lección

Dado un conjunto pequeño de archivos o mensajes generados por un programa propio, formular y comprobar una estructura: campos, offsets, longitudes, endianness, framing y estados. Distinguir checksum, hash, MAC, compresión y cifrado como hipótesis diferentes, y anotar qué evidencia sostiene cada conclusión.

Prerrequisitos: Bits y endianness (1.1), formato PE (1.4), método de análisis (4.1), IPC de Windows como referencia. **Duración sugerida:** 6 horas.

Alcance y muestras

Usá archivos y programas **propios** o diseñados para la práctica. Para un protocolo, trabajá con dos procesos tuyos en una VM, una named pipe local o loopback. No envíes datos a servicios de terceros. **Capas:** conocimiento externo; ejemplo CLSF ilustrativo y sintético; laboratorio no verificado. La práctica propuesta usa programas propios y comunicación local.

Del síntoma a una pregunta comprobable

«¿Qué significa este blob?» es demasiado amplio. Convertí la duda en una pregunta que permita diseñar una prueba:

- ¿Los primeros cuatro bytes son una firma constante?
- ¿Este entero describe la longitud del cuerpo, del encabezado o del archivo completo?
- ¿Dos mensajes consecutivos se separan por longitud, delimitador o estado de la conexión?
- ¿Qué campo cambia cuando cambia solo el contenido?
- ¿El bloque final detecta corrupción accidental, autentica el mensaje o solo parece aleatorio?

Conservá cada muestra original sin modificarla, calculá su hash y anotá cómo se obtuvo. Trabajá sobre copias. Una tabla de diferencias entre muestras vale más que una interpretación basada en un solo archivo.

Procedimiento de reconstrucción

1. **Inventario.** Anotá tamaño, hash, origen de laboratorio, herramienta que lo produjo y si el espécimen es archivo, mensaje, flujo o captura. Identificá la envoltura conocida y separala del formato interno.
2. **Muestras controladas.** Generá pares que difieran en una sola propiedad: una cadena más larga, otro valor, un registro adicional o una versión distinta. Incluí casos válidos vacíos y límites que tu programa admita.
3. **Mapa de bytes.** Numera offsets desde cero; registra bytes, ASCII/UTF-16, enteros little-endian/big-endian, alineación y regiones repetidas. No decidas el tipo de un campo por su apariencia aislada.

4. **Hipótesis.** Para cada campo escribe hipótesis, evidencia, alternativa y prueba que la refutaría. «Parece una longitud» no es conclusión hasta comprobar varios tamaños.
5. **Framing y estructura.** Separa cabecera, registros, cuerpo y tráiler. Prueba longitudes prefijadas, tamaños fijos, delimitadores, terminadores, conteos y esquemas TLV. Comprueba si las longitudes incluyen o excluyen cabecera/tráiler.
6. **Estado y secuencia.** Dibuja estados y transiciones: conexión, saludo, petición, respuesta, error y cierre. Relaciona cada mensaje con el estado previo; un opcode puede significar cosas distintas según el estado.
7. **Transformaciones.** Investiga por separado codificación, compresión, checksum/CRC, hash, MAC y cifrado. Usa perturbaciones controladas y evidencia del programa propio; entropía alta por sí sola no demuestra cifrado ni compresión.
8. **Validación independiente.** Comprueba la hipótesis con muestras no usadas para formularla, límites y entradas malformadas inocuas. Escribe un esquema y un parser de referencia mínimo.

Endianness, longitudes y TLV

Un entero de dos bytes 05 00 vale 5 en little-endian; 00 05 vale 5 en big-endian. En tráfico IP, los campos numéricos de Winsock usan orden de red (big-endian), aunque el host x86/x64 use little-endian. Un formato puede mezclar órdenes: documenta el orden **por campo**.

En un esquema **TLV** (type, length, value), cada elemento indica tipo, longitud y valor. Confirma el ancho de tipo/longitud, si la longitud incluye el encabezado y cómo se representa un elemento desconocido.

Ejemplo trabajado: formato sintético CLSF

*(Ejemplo **ilustrativo**, generado para la lección; no proviene de un archivo CLS.)* Supongamos un programa propio que escribe estos bytes:

```
Offset  Bytes                                   Lectura provisional
00      43 4C 53 46                              firma ASCII «CLSF»
04      01                                       versión candidata = 1
05      00                                       flags candidatos = 0
06      0C 00                                    tamaño de cabecera candidato = 12 (LE)
08      05 00                                    tamaño de cuerpo candidato = 5 (LE)
0A      2A 00                                    campo variable = 42 (LE)
0C      48 45 4C 4C 4F                           cuerpo ASCII «HELLO»
```

La muestra completa tiene 17 bytes. Es tentador llamar «secuencia» al campo en offset 0A, pero una sola muestra no lo justifica. Generamos variantes cambiando una cosa cada vez:

Muestra	Cambios controlados	Observación
A	Cuerpo HELLO, campo 2A 00	Longitud 5; total 17 bytes
B	Mismo cuerpo, campo 2B 00	Solo cambia el campo en 0A; podría ser contador, ID o checksum
C	Cuerpo BYE!!, campo 2A 00	Cambia solo el cuerpo; el campo 0A permanece
D	Cuerpo HELLO!!!, campo 2A 00	Cuerpo crece tres bytes y el campo 08 pasa de 5 a 8

A y D apoyan la hipótesis «offset 08 es longitud del cuerpo» y que empieza en 0C. A/B/C sugieren que 0A no es un checksum simple del cuerpo, pero no identifican su significado. Solo al observar varias operaciones del programa propio —por ejemplo, que incrementa entre mensajes— podríamos etiquetarlo como contador. Cada etiqueta queda como hipótesis hasta probarla.

Framing y máquina de estados

En un archivo, el inicio y final vienen dados por el archivo. En un protocolo hay que averiguar **dónde termina cada mensaje**:

- **Tamaño fijo:** cada mensaje tiene la misma longitud.
- **Longitud prefijada:** un encabezado anuncia cuántos bytes siguen; determina si cuenta desde el encabezado o solo el cuerpo.
- **Delimitador:** un byte o secuencia separa mensajes; prueba cómo se escapa si aparece en el cuerpo.
- **Estado o longitud implícita:** el final depende de una operación o fase del protocolo.

TCP es un flujo de bytes: los límites de una llamada send no son automáticamente límites de mensaje para quien recibe. El framing pertenece al protocolo de aplicación. Named pipes de Windows pueden operar en modo byte o mensaje; identifica el modo antes de interpretar lecturas, y documenta también la estructura interna definida por la aplicación.

```
DESCONECTADO --conectar--> ESPERA_SALUDO
ESPERA_SALUDO --saludo válido--> LISTO
LISTO --petición--> ESPERA_RESPUESTA
ESPERA_RESPUESTA --respuesta/error--> LISTO
cualquier estado --cierre--> DESCONECTADO
```

El diagrama es una hipótesis hasta contrastarla con trazas de una ejecución propia. Anota estado previo, bytes, respuesta y efecto observable para cada transición.

No confundir checksum, hash, MAC, compresión y cifrado

- **Checksum/CRC:** detecta ciertas corrupciones accidentales; no autentica al emisor.
- **Hash:** resumen de tamaño fijo; sin secreto no prueba quién creó el mensaje.
- **MAC:** etiqueta con clave compartida que permite verificar integridad/autenticidad bajo ese secreto.
- **Cifrado:** transforma datos usando una clave para ocultar contenido; no implica por sí mismo integridad.
- **Compresión:** transforma la representación para reducirla; firmas y cambios de tamaño dan pistas, pero entropía alta no prueba compresión ni cifrado.
- **Codificación:** cambia la representación (por ejemplo, base64 o UTF-16) sin ser cifrado.

Cambia un solo byte y compara el tráiler en muchas muestras. Después cambia una propiedad semántica y repite. Si no puedes validar una fórmula o identificar una llamada concreta en el programa propio, registra «campo no resuelto» en vez de adivinar CRC/AES.

Práctica

En una VM, con **un escritor/parser propio** y, si eliges la rama de protocolo, dos procesos tuyos:

1. Genera al menos cuatro muestras con una sola variable distinta por par. Conserva originales y hashes.
2. Completa una tabla de offsets, ancho, orden de bytes, valor observado, hipótesis y confianza.
3. Identifica el framing y dibuja una máquina de estados para un intercambio local. Si usas named pipe, registra su modo byte/mensaje.
4. Investiga un posible tráiler variando un byte y el contenido; distingue qué puedes concluir y qué no sobre checksum, hash, MAC, cifrado o compresión.
5. Escribe un esquema breve y un parser de referencia para muestras válidas. Añade límites y casos malformados inocuos.
6. Reserva dos muestras para validación: predice sus offsets, longitudes y transiciones antes de abrirlas.

Entregable

Un mini-informe con hashes, tabla de campos/offsets, diagrama de estados si es protocolo, esquema, pruebas, hipótesis refutadas y límites. Marca cada campo como **confirmado**, **probable** o **sin resolver**, con su evidencia.

Preguntas de repaso

▼ **¿Por qué un mensaje TCP no termina necesariamente donde terminó la llamada send?**

> TCP entrega un flujo de bytes; el protocolo de aplicación define el framing. Los segmentos y llamadas de envío no garantizan los límites que verá cada lectura.

▼ **¿Qué experimento ayuda a identificar un campo de longitud?**

> Cambiar solo el tamaño del cuerpo en varias muestras y comprobar si el campo cambia coherentemente y si cuenta encabezado o archivo completo.

▼ **¿Que un tráiler cambie al modificar el cuerpo demuestra que es un CRC?**

> No. También podría ser hash, MAC, cifrado u otro campo dependiente del contenido. Hay que probar la hipótesis con más muestras y evidencia.

▼ **¿Qué diferencia hay entre minimizar una muestra y confirmar un formato?**

> Minimizar reduce una entrada manteniendo un comportamiento o cobertura; confirmar exige que una hipótesis estructural prediga muestras independientes y casos límite.

Puerta de salida

Podés reconstruir un formato o protocolo propio desde muestras controladas: marcar campos y offsets, explicar endianness/longitudes/framing, dibujar estados cuando corresponda, separar transformaciones candidatas y validar el esquema con muestras reservadas. No hace falta adivinar todos los campos; sí declarar qué evidencia falta.

Fuentes

- Microsoft Learn: Byte Ordering, Named Pipes, Named Pipe Type, Read, and Wait Modes.
- IETF: RFC 9293, Transmission Control Protocol (TCP).
- Método de análisis, IPC en Windows y fuzzing/análisis simbólico.

Reconocer criptografía y encoding en un binario

> **De dónde sale esta lección**
>
> Este tema **no está en el corpus CLS**: es **conocimiento externo** que agregamos para completar la guía. Se incluye porque reconocer cripto y encoding es rutina en análisis de malware de Windows y en CTF. El enfoque es **defensivo y de identificación**: reconocer qué algoritmo hay, no implementar criptografía ni romperla.

> **Objetivo de la lección**
>
> Al terminar vas a poder distinguir encoding de cifrado, reconocer un XOR y un base64 en un listado, e identificar algoritmos conocidos (AES, RC4, MD5/SHA, TEA, CRC32) por sus **constantes y tablas**.

Prerrequisitos: instrucciones, flujo, método (Etapas 2 y 4). **Duración sugerida:** 3 horas.

Encoding no es cifrado

Es la primera distinción, y la que más se confunde:

- **Encoding** (base64, hex, URL-encoding) transforma datos para transportarlos. **No tiene clave**: cualquiera lo revierte. Si algo está en base64, está "ofuscado", no "cifrado".
- **Cifrado** usa una **clave**. Sin la clave no se recupera el texto (salvo error de implementación).
- **Hashing** (MD5, SHA) es de **una sola vía**: no se "descifra". Se usa para comparar o verificar.

Para el análisis, la pregunta no es "¿cómo lo rompo?", sino "¿qué transformación es y con qué parámetros?".

XOR: el disfraz más común

El XOR byte a byte contra una clave fija es el modismo número uno para esconder strings en malware:

```
loop:
    mov   al, [esi]
    xor   al, 0x5A        ; clave de 1 byte
    mov   [edi], al
    inc   esi
    inc   edi
    loop  loop
```

Señales de XOR:

- Un bucle corto con `xor` contra una constante o contra bytes de otra zona.
- Strings que **no aparecen** con `strings` / `Shift+F12` pero que el programa "tiene" (están XOR-eadas y se desXOR-ean en runtime).

- Datos en `.data` que parecen basura hasta que corre una rutina.

El XOR de 1 byte es trivial de deshacer (hay 255 claves posibles). Para análisis, muchas veces alcanza con un breakpoint **después** del bucle para leer el texto ya desXOR-eado en memoria, sin tocar el algoritmo.

Base64: la tabla que lo delata

Base64 se reconoce por su **alfabeto** de 64 caracteres, que casi siempre aparece como un string en `.rdata`:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
```

Más el `=` de relleno. Si ves ese alfabeto (o una variante con `-_` en vez de `+/`, que es base64 "url-safe"), hay base64. Procesa de a **4 caracteres** → **3 bytes**. Un alfabeto reordenado es base64 "custom": misma mecánica, otra tabla.

Algoritmos conocidos: las constantes los delatan

Casi todos los algoritmos famosos tienen **constantes o tablas fijas**. Encontrarlas es identificar el algoritmo sin leer una sola instrucción de la rutina:

Algoritmo	Qué buscar
AES (Rijndael)	La S-box, que empieza `63 7C 77 7B F2 6B 6F C5 30 01 67 2B …`; o las T-tables
MD5	Init `67452301 EFCDAB89 98BADCFE 10325476`; tabla de senos `D76AA478 …`
SHA-1	Init `67452301 EFCDAB89 98BADCFE 10325476 C3D2E1F0`
SHA-256	Init `6A09E667 BB67AE85 3C6EF372 …`; constantes `428A2F98 71374491 …`
RC4	Un arreglo de **256 bytes** inicializado `0,1,2,…,255` y un bucle de *swaps* (KSA/PRGA)
TEA / XTEA	La constante *delta* `0x9E3779B9` (número áureo)
CRC32	El polinomio `0xEDB88320` o una tabla de 256 entradas de 32 bits
DES	Tablas de permutación fijas (IP, S-boxes)

Un bucle que recorre un arreglo de 256 bytes haciendo intercambios casi seguro es **RC4**. Cuatro valores `67452301…` juntos casi seguro son **MD5 o SHA-1**. No hace falta entender el algoritmo: alcanza con nombrarlo y buscar su especificación.

Herramientas que automatizan el reconocimiento

- **FindCrypt / findcrypt-yara** (plugin de IDA/Ghidra): busca estas constantes y te dice qué algoritmo hay.
- **signsrch**: firmas de cientos de algoritmos.
- **CAPA** (Mandiant): identifica capacidades, incluida criptografía, por patrones.
- **YARA** con reglas de cripto: para clasificar muestras.

- **Detect It Easy / entropía**: una sección con entropía muy alta (cercana a 8 bits/byte) suele estar **cifrada o comprimida/empaquetada**; es una señal, no una prueba.

> **En Windows, muchas veces no hay que reconocer nada**
>
> Antes de pelear con una rutina de cripto, mirá los **imports**: si el programa usa la criptografía del sistema (CryptoAPI en `advapi32.dll` — `CryptEncrypt`, `CryptHashData` — o CNG en `bcrypt.dll` — `BCryptEncrypt` —), el algoritmo y la clave pasan por esas APIs. Un breakpoint en esas funciones te muestra los datos en claro sin reimplementar nada. El trabajo de reconocer constantes queda para cuando el programa trae su **propia** implementación embebida.

Un método de reconocimiento

1. Corré `strings` y FindCrypt/CAPA primero: muchas veces resuelven la identificación en segundos.
2. Si hay strings que "faltan", buscá un bucle con `xor` y poné un breakpoint después para leerlas en claro.
3. Si hay un alfabeto de 64 caracteres, es base64; mirá si es estándar o custom.
4. Si hay constantes de la tabla de arriba, nombrá el algoritmo y buscá su especificación.
5. Documentá **qué** algoritmo es y **con qué parámetros** (clave embebida, IV, tabla custom). No necesitás reimplementarlo para el informe.

> **Marco de uso**
>
> Identificar criptografía para entender un programa, detectar malware o resolver un CTF es análisis legítimo. Usar esto para romper el cifrado de software o datos ajenos, o para producir malware, no entra en esta guía. El objetivo es **reconocer y documentar**, no vulnerar.

> **Práctica**
>
> 1. Escribí un programa propio que guarde una string XOR-eada con una clave de 1 byte y la desXOR-ee al arrancar. Analizalo: encontrá el bucle `xor` y leé la string en memoria después.
> 2. Meté el alfabeto base64 en un programa y comprobá que aparece como string en `.rdata`.
> 3. Bajá una implementación de referencia de MD5 (de código abierto), compilala y buscá las constantes `67452301…` en el binario con tu desensamblador.
> 4. Corré FindCrypt o CAPA sobre esos binarios propios y compará con lo que encontraste a mano.

Preguntas de repaso

<details>
<summary>¿Cuál es la diferencia entre encoding y cifrado?</summary>

El encoding (base64, hex) no tiene clave y cualquiera lo revierte; el cifrado usa una clave y sin ella no se recupera el texto. El hashing, además, es de una sola vía.
</details>

▼ **¿Cómo reconocés base64 en un binario?**

> Por su alfabeto de 64 caracteres (`A–Z a–z 0–9 + /`) en `.rdata` y el `=` de relleno; procesa de a 4 caracteres → 3 bytes.

▼ **Encontrás un arreglo de 256 bytes inicializado `0,1,2,…` y un bucle de intercambios. ¿Qué algoritmo es probable?**

> RC4 (su fase de inicialización, KSA).

▼ **Ves las constantes `67452301 EFCDAB89 98BADCFE 10325476`. ¿Qué sugieren?**

> MD5 o SHA-1 (comparten ese estado inicial). Hay que mirar el resto para distinguirlos.

Antes de avanzar

Tenés que poder:

- Distinguir encoding, cifrado y hashing.
- Reconocer un XOR y un base64 en un listado.
- Identificar un algoritmo conocido por sus constantes/tablas.
- Usar FindCrypt/CAPA y la entropía como ayuda.

> **Puerta de salida**
>
> Si podés mirar un binario, decir "acá hay base64 y acá un XOR con clave de 1 byte, y esta tabla es la S-box de AES" y documentarlo, cumpliste el objetivo.

Fuentes de esta lección

- *Practical Malware Analysis* (Sikorski & Honig), capítulo de criptografía: XOR, base64 y reconocimiento de algoritmos.
- Proyectos **FindCrypt**, **signsrch** y **CAPA** (Mandiant) como referencia de firmas.
- Especificaciones públicas: FIPS 197 (AES), RFC 1321 (MD5), FIPS 180-4 (SHA), RFC 4648 (base64).

> **Capas de esta lección**
>
> **Conocimiento externo (agregado por esta edición):** todo el contenido (XOR, base64, constantes de algoritmos, CryptoAPI/CNG, herramientas). No proviene de CLS. **Recomendación editorial:** el método de reconocimiento y el encuadre defensivo.

Ampliación editorial basada en fuentes externas — Reconocimiento de ChaCha20 / Salsa20 (RFC 8439) por sus constantes y operaciones ARX

En binarios modernos de Windows, el cifrado de flujo **ChaCha20** (IETF RFC 8439) es sumamente frecuente y no utiliza tablas S-box *(Hecho confirmado)*:

- **Constante mágica de 16 bytes (clave de 256 bits):** La cadena ASCII `"expand 32-byte k"` o los cuatro `DWORD` little-endian `0x61707865`, `0x3320646e`, `0x79622d32`, `0x6b206574` en las palabras `0..3` de un estado de 16 palabras (64 bytes). (Para clave de 128 bits: `"expand 16-byte k"`).
- **Firma en el código (Quarter-Round ARX):** Un bucle de 20 rondas (10 iteraciones dobles) compuesto exclusivamente por sumas de 32 bits (`add`), `xor` y rotaciones (`rol`) con conteos de **16, 12, 8 y 7 bits** (o 7, 9, 13 y 18 en Salsa20).

Ampliación editorial basada en fuentes externas — Criptografía asimétrica (RSA/ECC) y tabla de intercepción en APIs nativas de Windows (CryptoAPI y CNG)

El software moderno rara vez implementa algoritmos criptográficos desde cero; en su lugar, utiliza esquemas híbridos apoyados en las bibliotecas criptográficas nativas de Windows (Microsoft Learn: Cryptography Next Generation y CryptoAPI):

1. Esquemas Criptográficos Híbridos en la Práctica

La carga útil o los datos de configuración se cifran con un algoritmo simétrico de alta velocidad (AES-CBC, AES-GCM o ChaCha20) utilizando una clave de sesión generada al vuelo. A continuación, esa clave de sesión se cifra con una **clave pública asimétrica** (RSA o Curvas Elípticas ECC) embebida en el binario. Solo quien posee la clave privada correspondiente puede descifrar la clave de sesión y acceder a los datos.

2. Reconocimiento de Claves Asimétricas en Binarios

- **RSA:** Se detecta por la presencia del exponente público estándar `e = 65537` (`0x00010001`, codificado en little-endian como `01 00 01 00` en memoria) y módulos grandes `N` de 256 bytes (2048 bits) o 512 bytes (4096 bits) en `.rdata`.
- **Estructura `RSAPUBKEY` (CryptoAPI clásica):**

```
struct RSAPUBKEY {
    DWORD magic;  // 0x31415352 = ASCII "RSA1" (clave pública) / 0x32415352 = "RSA2"
(privada)
    DWORD bitlen; // Tamaño en bits (ej. 0x00000800 = 2048 bits)
    DWORD pubexp; // Exponente público (ej. 0x00010001 = 65537)
};
```

- **Curvas Elípticas (ECC / ECDSA / ECDH):** Se identifican por constantes de curvas estándares como `secp256r1` (NIST P-256) o `Curve25519` (constante de campo primo `2^255 - 19`).

3. Tabla de APIs Criptográficas Nativas y Puntos de Intercepción en Depuración

Operación	CryptoAPI Clásica (advapi32.dll)	CNG Moderna (bcrypt.dll)	Estrategia de Intercepción en Depurador
Inicialización	`CryptAcquireContextW`	`BCryptOpenAlgorithmProvider`	Identifica el algoritmo pasando cadenas como `BCRYPT_AES_ALGORITHM` o `BCRYPT_RSA_ALGORITHM`.
Importación de Clave	`CryptImportKey`	`BCryptImportKeyPair` / `BCryptGenerateSymmetricKey`	Poner breakpoint para volcar el buffer de la clave en memoria (estructura `BLOBHEADER` o clave simétrica cruda).
Cifrado de Datos	`CryptEncrypt`	`BCryptEncrypt`	**Punto crítico de intercepción:** Poner breakpoint en la entrada. El argumento `pbData` (CryptoAPI) o `pbInput` (`RDX` en x64 para CNG) contiene el **texto plano antes de ser cifrado**.
Descifrado de Datos	`CryptDecrypt`	`BCryptDecrypt`	Poner breakpoint en el **retorno de la función**. El buffer de salida contendrá el texto descifrado en memoria.

Ampliación editorial basada en fuentes externas — La trampa del Endianness en CNG vs. CryptoAPI y script de conversión en Python

Una de las discrepancias arquitectónicas más frecuentes al extraer claves criptográficas de memoria radica en la convención de orden de bytes (Microsoft Learn: CNG Key Storage):

Característica	CryptoAPI Clásica (advapi32.dll)	CNG Moderna (bcrypt.dll / ncrypt.dll)
Estructura del encabezado	`PUBLICKEYSTRUC` (`BLOBHEADER`) + `RSAPUBKEY`	`BCRYPT_RSAKEY_BLOB` / `BCRYPT_ECCKEY_BLOB`
Magic Bytes de clave pública RSA	`0x31415352` (ASCII `"RSA1"`)	`0x31415352` (`BCRYPT_RSAPUBLIC_MAGIC` = `"RSA1"`)
Magic Bytes de clave privada RSA	`0x32415352` (ASCII `"RSA2"`)	`0x32415352` (`BCRYPT_RSAPRIVATE_MAGIC` = `"RSA2"`)
Magic Bytes de Curvas Elípticas (P-256)	No soportado de forma estándar	`0x314B4345` (`"ECK1"` para ECDH) / `0x31534345` (`"ECS1"` para ECDSA)
Orden de Bytes (Endianness) del Módulo y Claves	**Little-Endian** (el byte menos significativo va primero)	**Big-Endian** (formato estándar de red / RFC)

Reconocimiento Visual de Claves ASN.1 DER (X.509)

Cuando un binario no utiliza APIs nativas de Windows y compila su propia librería (como mbedTLS u OpenSSL), las claves públicas suelen embeberse en formato ASN.1 DER binario. Se reconocen al instante por la secuencia invariable que codifica el OID de `rsaEncryption` (`1.2.840.113549.1.1.1`):

```
30 82 .. .. 30 0D 06 09 2A 86 48 86 F7 0D 01 01 01 05 00 03 82 .. .. 00 30 82 ..
```

Script de Extracción y Conversión a PEM (Python / PyCryptodome)

Para convertir un volcado hexadecimal obtenido en x64dbg de una estructura `RSAPUBKEY` (CryptoAPI) a una clave pública `.pem` estándar utilizable en scripts defensivos, se debe invertir el orden de los bytes del módulo:

```
# Reconstrucción de clave pública RSA desde volcado de CryptoAPI (Little-Endian)
from Crypto.PublicKey import RSA

def parse_cryptoapi_rsapubkey(data: bytes) -> RSA.RsaKey:
    # data: buffer que comienza en RSAPUBKEY (después del BLOBHEADER de 8 bytes)
    magic = data[0:4]          # b'RSA1' (0x31415352)
    bitlen = int.from_bytes(data[4:8], 'little')
    pubexp = int.from_bytes(data[8:12], 'little')
    modlen = bitlen // 8
    # En CryptoAPI el módulo viene en little-endian; se invierte a big-endian:
    modulus = int.from_bytes(data[12:12+modlen][::-1], 'big')
    return RSA.construct((modulus, pubexp))

# Exportar a PEM: print(key.export_key().decode())
```

El informe reproducible

> **Objetivo de la lección**
>
> Al terminar vas a poder escribir un informe de análisis con entorno, hashes, método, evidencia, conclusión y límites, y revisar el informe de otra persona con una lista de verificación.

Prerrequisitos: método de análisis. **Duración sugerida:** 3 horas. Estimación aproximada.

Para qué sirve un informe

Un análisis sin registro es una anécdota. Sirve para vos mientras lo recordás; no sirve para un equipo, no se puede auditar y no se puede repetir. El informe convierte una sesión de debugger en **conocimiento verificable**.

Hay tres destinatarios posibles:

1. **Vos mismo, en dos semanas.** ¿Qué binario era? ¿Con qué versión? ¿Qué probaste?
2. **Otra persona del equipo.** ¿Puede repetir el experimento sin preguntarte nada?
3. **Un tercero que necesita decidir.** ¿Es seguro? ¿Hay un riesgo? ¿Qué mitigación recomendarías?

El nivel de detalle cambia según el destinatario, pero la estructura es la misma.

Estructura mínima

```
1. Identificación
   - Archivo, tamaño, SHA-256, origen, fecha de recepción
   - Entorno: SO, VM, herramientas y versiones, snapshot
2. Alcance y pregunta
   - Qué se analizó y qué quedó fuera
   - Pregunta concreta que guía el trabajo
3. Método
   - Estático: herramienta, pasos, direcciones
   - Dinámico: breakpoints, valores, condiciones
4. Observaciones
   - Tabla de evidencia: qué se hizo, qué se observó, dónde está la prueba
5. Conclusión
   - Respuesta directa, con grado de certeza
6. Límites
   - Qué no se probó, qué supuestos quedan, qué podría cambiar el resultado
7. Recomendaciones
   - Defensivas: detección, mitigación, monitoreo (si corresponde)
8. Anexos
   - Logs, capturas, scripts, hashes de artefactos
```

La plantilla del proyecto está en `plantilla_informe.md`. Esta lección la amplía con criterios de evidencia.

Ejemplo: informe corto de `check.exe`

Este es un informe de ejemplo sobre el programa de práctica de la lección anterior. Los valores son los que se observaron en esa práctica; las direcciones son ilustrativas y pueden variar en tu compilación.

```
INFORME DE ANÁLISIS — check.exe
Fecha: 2026-10-26
Analista: (tu nombre)

1. IDENTIFICACIÓN
Archivo: check.exe (52 KB)
SHA-256: <hash calculado con Get-FileHash>
Origen: compilado por el analista a partir de check.c (gcc -O0 -g)
Entorno: Windows 11 en VM, snapshot 00-limpia, x64dbg 2025.x, IDA Free 8.x
Alcance: determinar la condición de aceptación de dos números enteros.
Fuera de alcance: comportamiento fuera de la VM, otras compilaciones.

2. MÉTODO
Estático: búsqueda de la cadena "Correcto" (Shift+F12), xref (X),
lectura del grafo, renombrado de variables.
Dinámico: breakpoint en la comparación final, ejecución con
(40,80), (40,70) y (60,80).

3. OBSERVACIONES
| Entrada | EAX en la comparación | ZF | Salida |
| 40, 80  | 0x1E0                 | 1  | Correcto |
| 40, 70  | 0x1B8                 | 0  | Incorrecto |
| 60, 80  | no llega              | —  | Incorrecto (falla a>50) |

4. CONCLUSIÓN
El programa acepta pares (a,b) con a <= 50, b >= 50 y (a+b)*4 == 480.
Certeza: alta para esta compilación; no se probó exhaustivamente el espacio.

5. LÍMITES
No se analizaron otras compilaciones ni variantes del código.
No se evaluó el comportamiento con entradas no numéricas.
```

Ese informe es corto, pero responde: qué, cómo, con qué, qué se vio y qué falta.

Qué hace que una evidencia sea buena

Buena evidencia	Mala evidencia
Valor exacto, dirección, timestamp	"Se ve que compara"
Captura con la ventana y el valor visibles	Captura recortada sin contexto
Comando reproducible	"Lo hice con una herramienta"
Hash del binario	"El archivo que me pasaron"
Diferencia entre observado y supuesto	Mezcla de ambos sin aclarar

Una captura vale si se entiende **qué** muestra y **dónde** está el dato. Si recortás la ventana y no se ve la dirección ni el registro, no sirve como evidencia.

Trazabilidad: el hash y la cadena de custodia

El SHA-256 del binario es la forma de decir "este análisis corresponde exactamente a este archivo". Si el archivo cambia, el hash cambia.

Buenas prácticas:

1. Calculá el hash **al recibir** el archivo y anotalo.
2. Guardá una copia de trabajo con permisos de lectura.
3. No modifiques el original; si parcheás, trabajá sobre una copia y anotá el hash de la copia.
4. Registrá quién tuvo acceso y cuándo, si el contexto lo requiere.

Para malware, la cadena de custodia es más estricta: el archivo no sale del laboratorio, no se comparte por correo ni se sube a servicios públicos sin evaluar el riesgo.

Cómo se redacta la conclusión

Una conclusión no es un resumen del método. Es una **respuesta**. Tiene que poder leerse sola.

Mal	Bien
"Se analizó el binario y se encontraron comparaciones."	"El programa acepta `a <= 50`, `b >= 50` y `(a+b)*4 == 480`."
"Parece que hay un chequeo."	"El chequeo está en la comparación de `EAX` con `0x1E0`; con (40,80) `ZF=1`."
"Es seguro."	"No se observó actividad de red ni escritura fuera de la carpeta de trabajo en la VM."

La segunda columna es más útil porque **afirma algo que se puede contradecir con evidencia**.

Divulgación responsable

Si en un análisis autorizado encontrás una vulnerabilidad:

1. **No publiques el exploit** ni los detalles técnicos antes de que el proveedor pueda responder.
2. Documentá el impacto, la versión afectada y los pasos mínimos para reproducirlo.
3. Contactá al proveedor por un canal privado; guardá la evidencia.
4. Acordá un plazo razonable (por ejemplo, 90 días) antes de publicar.
5. No accedas a datos de terceros, no escales privilegios y no dejes puertas abiertas.

Esta guía no enseña a explotar sistemas ajenos. Los ejercicios de vulnerabilidades se hacen en programas propios o laboratorios diseñados para eso.

Práctica

1. Escribí el informe completo de tu `check.exe` con la estructura de arriba.
2. Incluí una tabla de evidencia con al menos tres filas.
3. Agregá una sección de límites con dos supuestos que no verificaste.
4. Intercambiá el informe con un compañero (o releelo al día siguiente) y aplicá esta lista:

Punto	Sí/No
¿Está el hash del binario?	
¿Está el entorno y la versión de las herramientas?	
¿La pregunta es específica?	
¿Cada conclusión tiene evidencia asociada?	
¿Se distingue observado de supuesto?	
¿Se puede repetir el experimento con lo escrito?	
¿Los límites son honestos?	

Preguntas de repaso

1. ¿Por qué el hash es parte del informe? Porque identifica sin ambigüedad el archivo analizado. Si el hash cambia, el informe ya no aplica a ese archivo.

2. ¿Qué diferencia hay entre una observación y una conclusión? La observación es lo que viste ("EAX=0x1E0"); la conclusión es lo que inferís ("la condición es (a+b)*4=480").

3. ¿Para qué sirve la sección de límites? Para que el lector sepa qué no se probó y no sobreestime el alcance del análisis.

4. ¿Qué hace reproducible un informe? Que incluya el binario identificado por hash, el entorno, los pasos y los valores observados, de modo que otra persona pueda repetirlos.

5. ¿Qué se hace si se encuentra una vulnerabilidad? Documentarla, reportarla en privado al proveedor, no publicar el exploit y acordar un plazo de divulgación.

Antes de avanzar

Tenés que poder:

- Escribir un informe con las ocho secciones.
- Distinguir evidencia de opinión.
- Calcular y registrar el hash del binario.
- Explicar la diferencia entre divulgar y publicar un exploit.

Puerta de salida

Si tu informe puede repetirlo otra persona sin preguntarte nada, está listo. Si no, falta evidencia o falta método.

Fuentes de esta lección

- `plantilla_informe.md` — plantilla base del proyecto.
- `Inventario limpio, LEEME` — uso de SHA-256 y criterio de no ejecución.
- `Reversing desde cero, cap. 11` — método de análisis y documentación visual.
- `Reversing desde cero, cap. 15` — valores y confirmación que alimentan el informe de ejemplo.

Capas de esta lección

Contenido observado en CLS: el método de los capítulos 11 y 15 y el uso de hashes del inventario. **Conocimiento externo:** estructura de informes técnicos y divulgación responsable. **Recomendación editorial:** la plantilla ampliada, la tabla de evidencia y el informe de ejemplo.

Análisis dinámico seguro: cuándo ejecutar y cuándo no

> **Objetivo de la lección**
>
> Al terminar vas a poder decidir si un programa se analiza solo en estático o se ejecuta en un laboratorio, preparar la observación con herramientas de monitoreo y actuar si el comportamiento se sale de lo previsto.

Prerrequisitos: laboratorio (Etapa 0) y método (Etapa 4). **Duración sugerida:** 3 horas. Estimación aproximada.

La decisión más importante

No todo se ejecuta. Antes de abrir un programa en el debugger, hacete tres preguntas:

1. **¿Tengo permiso?** ¿Es mío, es de código abierto, es un crackme de práctica, o tengo autorización escrita?
2. **¿Puedo contenerlo?** ¿Está en una VM aislada, con snapshot, sin carpetas compartidas y sin red personal?
3. **¿Necesito ejecutarlo?** ¿La pregunta se responde con análisis estático, o el comportamiento dinámico es imprescindible?

Si alguna respuesta es "no", el programa no se ejecuta. Se analiza estático, se documenta y se deriva a quien corresponda.

Situación	Qué hacer
Programa propio o de práctica	Ejecutar en VM, con snapshot
Software abierto que querés entender	Ejecutar en VM, con la licencia respetada
Binario de origen desconocido	Estático primero; dinámico solo en laboratorio dedicado
Malware confirmado	Estático y detección; dinámico solo con protocolo defensivo
Software comercial con protección	No eludir protecciones; analizar solo lo permitido
Archivo en cuarentena o excluido	No ejecutar; respetar la clasificación

Antes de ejecutar: línea base

Una observación sin línea base no sirve. Antes de correr el programa, registrá el estado del sistema:

- Procesos en ejecución.
- Conexiones de red.
- Archivos en la carpeta de trabajo.
- Claves de registro relevantes (si vas a observar persistencia).
- Hora de inicio.

Herramientas: **Process Explorer** (procesos, handles), **Process Monitor** (archivos, registro, procesos), **TCPView** (conexiones), **Autoruns** (persistencia). Todas son de Sysinternals o equivalentes gratuitos.

En la VM:

```
1. Restaurar snapshot 00-limpia.
2. Anotar hora y estado.
3. Iniciar Process Monitor con filtro por nombre del proceso.
4. Iniciar TCPView.
5. Ejecutar el programa.
6. Observar, anotar y detenerse.
7. Cerrar el programa, detener las capturas y restaurar snapshot.
```

Qué observar

Categoría	Qué mirar	Herramienta
Archivos	Creación, lectura, escritura, borrado	Process Monitor
Registro	Claves creadas/modificadas	Process Monitor
Procesos	Hijos, inyecciones, elevación	Process Explorer
Red	Conexiones, DNS, puertos	TCPView, Wireshark
Memoria	Regiones RWX, módulos cargados	x64dbg, Process Explorer
Persistencia	Run keys, servicios, tareas	Autoruns

No hace falta observar todo a la vez. Elegí dos o tres categorías según la pregunta.

El caso Shelter: lo que CLS muestra y lo que no se copia

El video 37 analiza el malware Shelter dentro de una VM Windows 7 de 32 bits, con otra VM Kali Linux para simular el ataque. El propio autor distingue análisis estático y dinámico y dice que en su caso va a ejecutar la muestra (`Videos y material adicional, Video 37`, 00:24:01–00:24:21).

Ese video es un **caso observado**, no un modelo para esta guía. Ejecuta malware, monta un reverse shell y usa una segunda VM para el ataque. Para un principiante, el orden correcto es:

1. Triage estático: tipo de archivo, secciones, imports, strings, hashes.
2. Detección: reglas YARA, firmas, comparación con VirusTotal **sin subir muestras confidenciales**.
3. Documentación de indicadores.
4. Solo si hay un propósito defensivo claro y un laboratorio dedicado, análisis dinámico controlado.

El video 36 inyecta una puerta trasera de Metasploit en una calculadora y usa una VM "por seguridad" (`Videos y material adicional, Video 36`, 00:29:21–00:29:42). Tampoco es una práctica de esta guía.

> **Lo que no se hace**
>
> No se ejecuta malware en el host. No se conecta la VM de análisis a la red doméstica. No se comparten carpetas. No se usan cuentas personales. No se sube una muestra a un servicio público sin evaluar si es confidencial. No se copia el malware fuera del laboratorio.

Anti-debug: una señal, no un obstacle a vencer

Si un programa detecta el debugger y se cierra, eso **es información**. Puede indicar:

- Que el programa tiene protecciones anti-análisis.
- Que está empaquetado.
- Que su comportamiento cambia según el entorno.

Los chequeos más comunes están en el capítulo 22 (PEB, `IsDebuggerPresent`, `NtGlobalFlag`) y en el capítulo 24 (lista de procesos, ParentPID, `CloseHandle` inválido). El capítulo 24 usa un crackme propio que muestra qué técnica lo detectó (`Reversing desde cero, cap. 24`, 00:01:17–00:03:31).

Para esta guía:

- **Estudiá** los mecanismos para reconocerlos y explicar por qué un binario se comporta distinto.
- **No** aprendas a evadir las protecciones de software comercial.
- **No** uses técnicas de ocultamiento para analizar muestras fuera de un laboratorio autorizado.
- Si el objetivo es defender, la pregunta es "¿cómo detecto este comportamiento?", no "¿cómo lo esquivo?".

Si algo sale mal

Un plan de contención tiene que existir antes de ejecutar:

1. **Detener la VM** (no suspenderla): apagado forzado si hace falta.
2. **Desconectar la red** de la VM desde el hipervisor.
3. **Restaurar la snapshot** limpia.
4. **Revisar el host**: procesos, conexiones, archivos compartidos, cuentas.
5. **Anotar** qué pasó, a qué hora y qué se observó.
6. **Reportar** si hubo exposición de datos o credenciales.
7. **Cambiar** las credenciales que pudieron quedar expuestas.

Si el programa logró escribir en una carpeta compartida o conectar a la red local, el aislamiento falló. Eso se documenta y se corrige antes de la próxima práctica.

Práctica

1. Ejecutá tu `check.exe` en la VM con Process Monitor filtrando por el nombre del proceso. Anotá qué archivos abre y qué claves de registro lee.

2. Ejecutá una versión que abra un archivo (`fopen`) y observá la operación de escritura.
3. Hacé una lista de línea base antes y después. ¿Qué cambió?
4. Elegí un archivo del proyecto marcado `cuarentena` y hacé **solo** triage estático: tipo, secciones, imports, strings, hash. No lo ejecutes.
5. Escribí un procedimiento de 10 líneas para "qué hago si la VM se comporta raro".

Preguntas de repaso

1. ¿Cuándo NO se ejecuta un programa? Cuando no tenés permiso, no podés contenerlo o no necesitás el comportamiento dinámico. También cuando está en cuarentena o excluido.

2. ¿Por qué se toma una línea base antes de ejecutar? Porque sin el estado inicial no podés atribuir los cambios al programa. La comparación antes/después es la evidencia.

3. ¿Qué indica que un programa detecte el debugger? Que tiene protecciones anti-análisis o que su comportamiento depende del entorno. Es un dato para documentar, no un obstáculo para "saltar".

4. ¿Qué hace primero si una muestra resulta sospechosa? Triage estático, cálculo de hash, documentación de indicadores y, si corresponde, derivación. No se ejecuta en el host.

5. ¿Qué hace si la VM se conecta a la red local? Se desconecta desde el hipervisor, se restaura la snapshot, se revisa el host y se documenta el fallo de aislamiento.

Antes de avanzar

Tenés que poder:

- Decidir si un programa se ejecuta o no, con justificación.
- Tomar una línea base y observar con Process Monitor/TCPView.
- Explicar por qué el video 37 no es un modelo de práctica.
- Describir un plan de contención.

> **Puerta de salida de la Etapa 4**
>
> Si podés analizar, documentar y contener una práctica completa, terminaste el recorrido base. Las ramas de la Etapa 5 son opcionales y exigen prerrequisitos.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — caso Shelter: VM, Kali y ejecución de malware; se cita como caso observado.
- `Videos y material adicional, Video 36` — XOR, crypter y backdoor en VM.
- `Reversing desde cero, cap. 22` — PEB y detección de debugger.
- `Reversing desde cero, cap. 24` — cuatro chequeos anti-debug.
- `Inventario limpio, LEEME` — estados `cuarentena` y criterio de no ejecución.

Capas de esta lección

Contenido observado en CLS: el laboratorio con dos VMs del video 37, la inyección del video 36 y los chequeos de los capítulos 22 y 24. **Conocimiento externo:** observación con Process Monitor/TCPView y planes de contención. **Recomendación editorial:** la matriz de decisión y el protocolo de incidentes.

Etapa 5 · Ramas opcionales

Objetivo medible

Elegir una especialización dentro de Windows, con prerrequisitos y práctica propia, sin convertir el corpus en una lista de tareas peligrosas.

Prerrequisitos: Etapas 0 a 4 completas. **Duración estimada:** Variable: 5 a 10 horas por rama. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
5.1	Rama malware: análisis defensivo sin ejecutar	Triage, IOCs, YARA y respuesta.
5.2	Rama vulnerabilidades: entender el defecto	Desbordes, mitigaciones de Windows y divulgación.
5.3	Rama automatización: scripts	IDAPython, Ghidra, pefile y capstone.
5.4	Kernel y otros territorios: qué queda fuera	Límites explícitos, otras plataformas y prerequisites.

Recursos de CLS para esta etapa

- 2D · Windows internals (rama optativa)
- Etapa 6 · Kernel y drivers (rama optativa, requiere base técnica específica)
- Exploiting (contenedores pendientes)
- Virus (excluido)
- Themida (excluido)

Evaluación de salida

- Elegir una rama y completar su práctica con documentación.
- Explicar los prerrequisitos y los límites de esa rama.
- No ejecutar muestras ni usar técnicas de evasión fuera del marco.

Entregable de la etapa

Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Saltar a exploiting sin dominar stack y mitigaciones.

- Ejecutar malware para "ver qué hace".
- Confundir un contenedor sin revisar con una fuente leída.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Rama malware: análisis defensivo sin ejecutar

> **Objetivo de la lección**
>
> Al terminar vas a poder hacer un triage estático de una muestra, extraer indicadores, escribir una regla YARA sencilla y explicar por qué el análisis dinámico de malware no es una práctica de principiante.

Prerrequisitos: etapas 0 a 4, PE y método. **Duración sugerida:** 5 horas. Estimación aproximada.

Qué hay en CLS y qué queda afuera

En `mega_OTROS` hay carpetas de "VIRUS / PROGRAMACION DE VIRUS" con material antiguo. Están fuera del recorrido base: no se leyeron, no se extrajeron y no se usan como práctica. En el inventario hay archivos en `cuarentena` que tampoco se ejecutan.

El video 37 analiza el malware Shelter **ejecutándolo** en una VM con Windows 7 y una VM Kali. Es un caso observado, no un modelo: esta rama hace lo contrario, empieza por el análisis estático y solo considera el dinámico en un laboratorio dedicado con propósito defensivo.

Por qué estático primero

Un programa malicioso está diseñado para hacer algo que no querés que pase en tu máquina. Ejecutarlo para "ver qué hace" es la forma más rápida de descubrirlo de la peor manera. El análisis estático responde muchas preguntas sin ejecutar:

- ¿Qué tipo de archivo es?
- ¿Está empacado?
- ¿Qué APIs importa?
- ¿Qué strings contiene?
- ¿Qué URLs, IPs o rutas menciona?
- ¿Se parece a una familia conocida?

Con eso ya podés escribir indicadores y reglas de detección. El dinámico se reserva para cuando el estático no alcanza y hay un laboratorio preparado.

Triage estático paso a paso

1. Identificación y hash

```
Get-FileHash -Algorithm SHA256 .\muestra.bin
```

Anotá el hash, el tamaño y el origen. Si el archivo está en cuarentena, no lo muevas de ahí.